Digital Hospital Management System

Project Report

Submitted by

Group Members:

KRISHNAJA K M (SR23CCSR06)

HARIKRISHNAN E (SR23CCSR12)

SREELAL A (SR23CCSR15)

SREENAND K(SR23CCSR16)

In partial fulfilment for the award of the Degree of

BSC COMPUTER SCIENCE

IN

DEPARTMENT OF COMPUTER SCIENCE

AT

SREE NARAYANA COLLEGE OF MANAGEMENT STUDIES PERIYA

SREE NARAYANA COLLEGE OF MANAGEMENT STUDIES PERIYA

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that the Project Report entitled "Digital Hospital Management System" submitted to the Department of Computer Science, Sree Narayana College of Management Studies, Periya, in partial fulfilment of the requirements for the award of the degree of Bachelor of Science in Computer Science (B.Sc Computer Science) is a record of the original work carried out by Group of 4 under our supervision and guidance.

This project work has not been submitted previously, in whole or in part, to any University or Institute for the award of any degree or diploma.

Project Co-Ordinator

Course Co-Ordinator

(Dep-office seal)

Internal Examiner

External Examiner

SREE NARAYANA COLLEGE OF MANAGEMENT STUDIES PERIYA

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

Certificate that this project "Digital Hospital Management System" is the Genuine work of Group of 4 who carried out the project work under my supervision

Shylaja

Project Guide

Assistant Professor

Department of Computer science

Sree Narayana College of Management Studies, Periye

DECLARATION

I hereby declare that the report of the project work entitled “Digital Hospital Management System” submitted to the Department of Computer Science, Sree Narayana College of Management Studies, Periya, in partial fulfilment of the requirement for the award of the degree of Bachelor of Science in Computer Science (BSc Computer Science) is a bonafide work carried out by the members of Group-4.

The material contained in this report has not been submitted to any University or Institute for the fulfilment of the requirements of any course of study.

ACKNOWLEDGEMENT

During the period that we worked on this project, we relied on the assistance, resources and support of many people. We acknowledge with gratitude the help, guidance and encouragement given by many individuals in the completion of this work.

We register our gratitude to our respected Principal Mrs. LATHA E S, for all the help and guidance.

We express our sincere thanks to Mrs. SHYLAJA C, Head of the Department of Computer Science and project guide, for her expert guidance and valuable advice during the development of this project.

We express our sincere thanks to Mrs. DEEPTHI G P, Mrs. RAMYA P V, and Ms. ARYA M, Lecturers in the Department of Computer Science, for their valuable support, encouragement, and guidance throughout the development of this project.

We are also thankful to the staff of the Department of Computer Science, Sree Narayana College of Management Studies, Periya , for their cooperation and support throughout the completion of this work.

We are very thankful to our parents and dear friends for their moral support.

Above all, we thank the Lord Almighty who gave us good health, courage and interest to complete this work successfully.

ABSTRACT

Abstract

This project presents a Hospital Management System (HMS) designed to modernize healthcare delivery by replacing error-prone paper records with a unified digital platform. The system addresses two critical challenges: illegible handwriting and fragmented communication between departments such as laboratories, pharmacies, and wards, both of which contribute to patient delays and inefficiencies.

The HMS integrates an HTML/CSS/JavaScript frontend with a robust Django (Python) backend, ensuring centralized access to patient records, secure prescriptions, and digitalized laboratory reports. By enabling real-time data sharing across departments, the system fosters coordinated care and minimizes administrative bottlenecks.

The primary objective is to reduce patient queues and waiting times by eliminating paperwork errors and streamlining hospital workflows, thereby saving valuable time for both patients and healthcare staff.

The system is structured around six core modules:

- Patient Management – Registration and maintenance of Electronic Health Records (EHR).

- Appointment & Scheduling – Efficient booking, tracking, and optimization of patient visits.

- Doctor/Physician Module – Access to EHRs, digital prescription issuance, and diagnosis recording.

- Laboratory Management – Digital test requests, sample tracking, and instant report publication.

- Pharmacy & Inventory Management – Processing prescriptions and monitoring drug stock levels.

- Billing & Administration – Automated invoice generation, payment tracking, and financial reporting.

Together, these modules form a comprehensive solution that enhances accuracy, efficiency, and patient satisfaction in hospital operations.

INDEX

NO	TITLE	PAGE
1	INTRODUCTION	**9**
2	SYSTEM STUDY	**12**
3	SYSTEM ANALYSIS	**15**
4	SYSTEM SPECIFICATIONS	**18**
5	SOFTWARE DESCRIPTION	**23**
6	PROJECT DESCRIPTION	**26**
7	SYSTEM TESTING	**44**
8	SYSTEM IMPLEMENTATION	**48**
9	CONCLUSION	**52**
10	SYSTEM SCREENSHOTS	**54**
11	FUTURE SCOPE	**63**
12	REFERANCE	**61**

1.INTRODUCTION

1.1 INTRODUCTION

The Hospital Management System (HMS) is a web-based application developed to modernize and digitalize hospital operations. Traditional hospital management methods rely heavily on paper-based records, manual appointment scheduling, handwritten prescriptions, and disconnected communication between departments. These practices often result in delays, miscommunication, data redundancy, and medical errors.

The proposed system replaces manual processes with a centralized digital platform developed using Django (Python) as the backend framework, MySQL as the database management system, and HTML, CSS, and JavaScript for the frontend interface.

The system integrates various hospital departments such as:

- Patient Management
- Appointment & Scheduling
- Doctor/Physician Module
- Laboratory Management
- Pharmacy & Inventory
- Billing & Administration

By enabling real-time data sharing between these modules, the system improves efficiency, accuracy, and coordination within the hospital.

The primary goal of the system is to:

- Reduce patient waiting time
- Eliminate errors caused by illegible handwriting
- Improve communication between departments
- Maintain secure and centralized Electronic Health Records (EHR)
- Automate billing and reporting

The system operates on a local server environment and provides role-based access control for different users such as administrators, doctors, laboratory staff, pharmacy staff, and billing personnel.

1.2 OBJECTIVE

The main objective of the Hospital Management System is to design and implement a centralized digital platform capable of managing all core hospital operations efficiently.

The specific objectives are:

- To maintain accurate patient records digitally
- To streamline appointment scheduling and management
- To enable doctors to issue digital prescriptions
- To allow laboratories to upload test reports electronically
- To facilitate pharmacy inventory management
- To automate billing and payment tracking
- To reduce paperwork and administrative overhead

The system aims to improve overall hospital workflow while enhancing patient satisfaction and operational transparency.

2.SYSTEM STUDY

2.1 EXISTING SYSTEM

In many hospitals, administrative and clinical operations are still managed manually or through partially digitized systems. Patient registration, appointment scheduling, prescription writing, laboratory test reporting, and billing processes are often handled using paper-based records or disconnected software tools.

Doctors write prescriptions manually, laboratory reports are maintained separately, pharmacy departments manage stock independently, and billing calculations are performed manually or using basic software without integration. This lack of coordination between departments leads to inefficiencies and delays.

2.1.1 DRAWBACKS OF EXISTING SYSTEM

The major drawbacks of the existing manual or semi-manual system are:

- Illegible handwritten prescriptions
- Delays in communication between departments
- Loss or misplacement of patient records
- Redundant data entry
- Increased patient waiting time
- Manual billing errors
- Difficulty in tracking pharmacy stock
- Lack of centralized data storage
- No real-time status updates

These issues negatively affect hospital efficiency and patient satisfaction.

2.2 PROPOSED SYSTEM

The proposed **Hospital Management System (HMS)** is a web-based centralized platform developed using **Django (Python)** and **MySQL**, with a frontend built using **HTML, CSS, and JavaScript**.

The system integrates all hospital departments into a single digital environment. Each module is role-based and accessible only to authorized users.

The proposed system includes:

- Digital patient registration and record maintenance
- Doctor access to patient history and digital prescription generation
- Laboratory test request management and report upload
- Pharmacy prescription processing and stock management
- Automated billing generation and status tracking
- Administrative monitoring and control

- All information is stored in a centralized MySQL database and accessed through a secure local server.

2.2.1 ADVANTAGES OF PROPOSED SYSTEM

- Centralized patient data management
- Reduced paperwork
- Faster communication between departments
- Digital prescriptions (no handwriting errors)
- Automated billing calculation
- Improved pharmacy inventory control
- Secure role-based access
- Better data consistency and accuracy
- Reduced patient waiting time

3.SYSTEM ANALYSIS

3.1 SYSTEM ANALYSIS

System analysis is the detailed study of the existing system to understand its limitations and to define the requirements of a new computerized system. It involves identifying user needs, evaluating feasibility, and designing an efficient solution.

In the context of the Hospital Management System, the analysis phase focused on identifying inefficiencies in manual hospital workflows and designing a centralized digital solution.

The main objectives of system analysis are:

- To identify hospital workflow problems
- To determine system requirements
- To evaluate feasibility
- To define database and module structure
- To ensure smooth integration between departments

The analysis was conducted in three stages:

1. Study of existing manual hospital processes
2. Feasibility analysis
3. Requirement specification

3.2 FEASIBILITY STUDY

Feasibility study determines whether the proposed system is practical and beneficial.

The Hospital Management System was evaluated under the following feasibility categories:

3.2.1 Economic Feasibility

Economic feasibility evaluates whether the system is cost-effective.

The proposed system:

- Uses open-source technologies (Python, Django, MySQL)
- Requires no licensing cost
- Runs on standard hardware
- Reduces paperwork expenses
- Minimizes administrative errors

Since the system reduces operational inefficiencies and does not require expensive infrastructure, it is economically feasible.

3.2.2 Technical Feasibility

Technical feasibility evaluates whether the system can be developed and implemented using available technology.

The system is built using:

- Python 3.14.2
- Django 4.2.11
- MySQL Database
- HTML, CSS, JavaScript

These technologies are stable, widely supported, and suitable for web-based applications. The system runs on a local server and does not require cloud infrastructure.

Hence, the system is technically feasible.

3.2.3 Operational Feasibility

Operational feasibility evaluates user acceptance and usability.

The system:

- Has role-based dashboards
- Provides structured digital forms
- Reduces manual paperwork
- Simplifies inter-department communication

Since the interface is simple and intuitive, hospital staff can easily adapt to the system after basic training.

Therefore, the system is operationally feasible.

4.SYSTEM SPECIFICATION

4.1 SYSTEM REQUIREMENTS

The successful operation of the Hospital Management System depends on appropriate hardware and software resources. Since the application is developed using Django (Python) with a MySQL database and a browser-based frontend, the system can run on standard computing devices without requiring high-end infrastructure.

The system is designed to be user-friendly, reduce redundancy, ensure secure access, and support efficient data handling across hospital departments.

4.1.1 HARDWARE REQUIREMENTS

For smooth performance, the following minimum hardware configurations are recommended:

Server / Development Machine Requirements

Component	**Minimum Requirement**
Processor	Intel i3 / AMD equivalent (Dual Core or above)
RAM	4 GB or higher
Storage	20 GB free disk space (SSD recommended)
Display	1366×768 resolution or above
Network	LAN/Wi-Fi (optional for multi-system usage)

Client System Requirements

Component	**Minimum Requirement**
Processor	Intel Dual Core / AMD equivalent
RAM	2 GB or higher
Storage	5 GB free disk space
Browser	Chrome / Firefox / Edge (latest versions)
Display	1366×768 resolution or above

4.1.2 SOFTWARE REQUIREMENTS

The software requirements specify the necessary platforms and tools needed to run the system.

Server / Development Side Requirements

Software	Requirement
Operating System	Windows 10/11 / Linux (Ubuntu)
Programming Language	Python 3.14.2
Framework	Django 4.2.11
Database	MySQL Server
IDE/Editor	VS Code / PyCharm (optional)
Browser	Chrome / Firefox / Edge

Client Side Requirements

Software	Requirement
Operating System	Any OS (Windows/Linux/Mac)
Browser	Chrome / Firefox / Edge
Internet	Not required (local server), LAN optional

4.1.3 TECHNOLOGY USED

The system is developed using the following technologies:

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Python (Django Framework)
- **Database:** MySQL
- **Server:** Django Development Server (Localhost)

4.1.4 SECURITY REQUIREMENTS

To ensure secure system access and data protection:

- Role-based authentication is implemented for Admin, Doctor, Lab, Pharmacy, and Billing staff.
- Only authorized users can access relevant modules.
- Data is stored securely in the MySQL database.
- Sensitive details such as user credentials are protected through controlled access.

4.2 IDENTIFICATION OF USE CASES

A use case represents a specific functionality provided by the system that results in a meaningful outcome for an external actor. In the Hospital Management System (HMS), different users interact with the system according to their assigned roles. Each actor is granted access only to the functionalities relevant to their responsibilities through role-based access control.

Identifying use cases involves analyzing how each user interacts with the system and determining the operations they are permitted to perform.

Actors in the System

The primary actors identified in the Hospital Management System are:

- Administrator
- Doctor
- Patient
- Receptionist
- Laboratory Staff
- Pharmacy Staff
- Billing Staff

Each actor performs a distinct set of operations within the system based on their role.

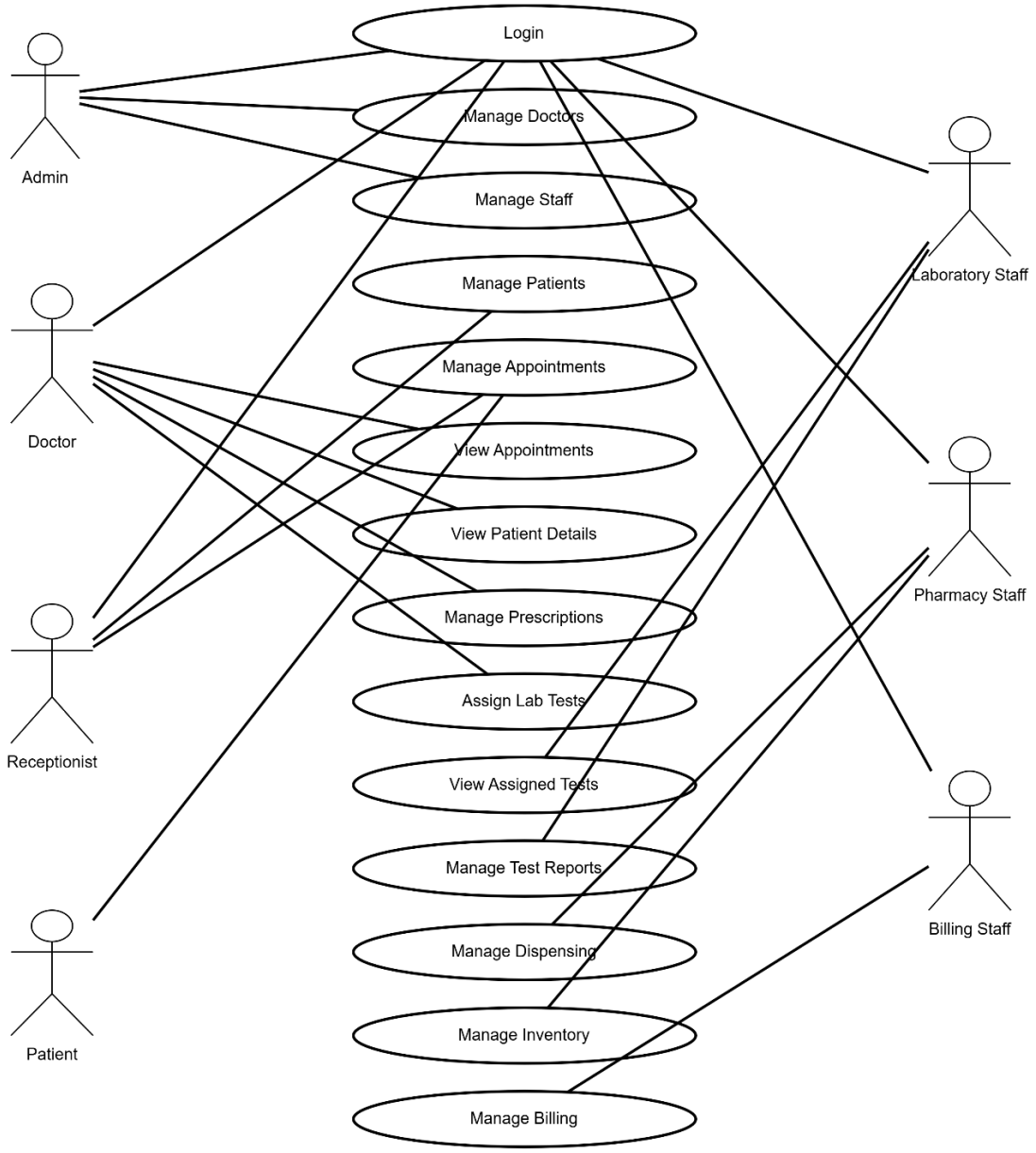
Approach Used to Identify Use Cases

The identification of use cases was guided by analyzing the functional responsibilities of each actor. The following questions were considered during this process:

- What operations does the actor perform within the system?
- Does the actor create, update, view, or delete data?
- Does the actor require notifications or status updates from the system?
- Can the actor's manual tasks be automated or simplified using the system?

By evaluating these factors, a structured set of use cases was derived for each module of the Hospital Management System.

4.3 USE CASE DIAGRAM



5.SOFTWARE DESCRIPTION

5.1 PYTHON

Python is a high-level, interpreted programming language known for its simplicity, readability, and wide library support. It is commonly used for web development, automation, data handling, and backend development.

In this project, Python is used to implement server-side logic such as:

- handling requests and responses,
- database operations,
- authentication (custom login tables),
- workflow control between modules.

Python enables rapid development and provides strong support for web frameworks like Django.

5.2 DJANGO FRAMEWORK

Django is a high-level Python web framework that supports rapid development and clean, pragmatic design. It follows the **Model-View-Template (MVT)** architectural pattern.

Django provides structured development through:

- Models for database mapping (ORM),
- Views for request handling and logic,
- Templates for rendering dynamic HTML pages.

In the Hospital Management System, Django is used to:

- implement module-based features (Admin, Doctor, Patient, Lab, Pharmacy, Billing),
- perform secure CRUD operations on hospital records,
- handle routing and form processing,
- connect and interact with the MySQL database efficiently.

Although Django provides a built-in authentication system, this project uses **custom login tables** to control access for Admin, Doctor, and Staff roles based on the module requirements.

5.3 MYSQL DATABASE

MySQL is a relational database management system that stores data in structured tables. It supports SQL queries, relationships, and constraints, making it suitable for large, organized datasets.

In this project, MySQL is used as the backend database to store:

- patient details,
- appointments and token scheduling,
- prescriptions and medicine instructions,
- laboratory tests and uploaded reports,

- pharmacy inventory,
- billing details and payment status.

The relational structure ensures data consistency and efficient retrieval across modules.

5.4 HTML (HYPERTEXT MARKUP LANGUAGE)

HTML is the standard markup language used to structure web pages. It defines elements such as headings, forms, tables, buttons, and navigation sections.

In this project, HTML is used to create:

- login pages,
- dashboards for each role,
- forms for prescriptions, tests, billing, and inventory,
- tables for listing patients, appointments, bills, and reports.

5.5 CSS (CASCADING STYLE SHEETS)

CSS is used to style and design HTML content. It controls the layout, colors, fonts, spacing, and responsiveness of the web pages.

In this project, CSS is used to:

- create clean dashboards,
- design tables and cards,
- provide consistent UI across modules,
- improve user experience by making pages more readable and professional.

5.6 JAVASCRIPT

JavaScript is a client-side scripting language used to make web pages interactive. It allows dynamic updates in the browser without reloading the page.

In this project, JavaScript is used for:

- form validations,
- interactive UI actions,
- button actions such as print and view toggles,
- improving the responsiveness and usability of the interface

6.PROJECT DESCRIPTION

6.1 PROBLEM DEFINITION

Hospitals often face challenges due to manual patient record management, handwritten prescriptions, disconnected departmental workflows, and inefficient billing processes. These issues result in:

- Increased patient waiting time
- Errors in prescriptions and billing
- Poor coordination between departments
- Data redundancy and record loss

The objective of this project is to develop a **web-based Hospital Management System (HMS)** that integrates all hospital departments into a single digital platform to improve operational efficiency and patient satisfaction.

6.2 OVERVIEW OF THE PROJECT

The Hospital Management System is a web-based application developed using Django and MySQL. The system provides a centralized platform where hospital activities such as patient registration, appointment scheduling, prescription generation, laboratory test management, pharmacy inventory handling, and billing operations are managed digitally.

The system supports the following workflow:

- Patients can register and book appointments independently.
- OTP verification is used for secure appointment viewing.
- Reception staff can also register patients and allocate appointment slots.
- Doctors can access patient records and generate digital prescriptions.
- Laboratory staff can upload test reports and update status.
- Pharmacy staff can dispense medicines and update inventory.
- Billing staff can generate invoices and track payment status.
- The administrator manages doctor and staff accounts.

The system operates on a local server and ensures role-based access control for secure usage.

6.3 MODULE DESCRIPTION

The system consists of seven main modules:

Admin Module

The Admin Module has full control over system management.

Functions:

- Admin login

- Add, view, and manage doctor accounts
- Add, view, and manage staff accounts
- Monitor system records
- Logout

The admin ensures smooth coordination across all departments.

Patient Module

This module handles patient registration and appointment booking.

Functions:

- Patient self-registration
- Appointment booking
- OTP generation and verification
- Viewing appointment details (Token & Time)
- Reception staff patient registration
- Reception staff appointment allocation

Patients do not require a permanent login account. OTP-based verification ensures secure access to appointment details.

Reception Module

This module helps reduce patient waiting time and ensures organized appointment scheduling.

Functions:

- Reception staff login
- Register new patients
- View patient details
- Edit or update patient information
- Book appointments for patients
- Assign doctor, date, token number, and appointment time
- View appointment list
- Reschedule appointments
- Cancel appointments
- Logout

The reception module ensures smooth communication between patients and doctors by efficiently managing patient records and appointment scheduling.

Doctor Module

The Doctor Module manages diagnosis and prescription processes.

Functions:

- Doctor login
- View scheduled appointments
- Access patient details
- Generate digital prescriptions
- Assign laboratory tests
- Update prescription status
- Logout

This module reduces handwriting errors and improves medical record tracking.

Laboratory Module

The Laboratory Module manages diagnostic tests.

Functions:

- Laboratory staff login
- View assigned tests
- Upload test report images
- Update test status (Assigned → Completed)
- Logout

This ensures real-time availability of diagnostic results.

Pharmacy Module

The Pharmacy Module handles medicine dispensing and stock management.

Functions:

- Pharmacy staff login
- View prescriptions
- Dispense medicines
- Update stock levels
- Manage medicine inventory (add/update price)
- Logout

The inventory system prevents stock mismanagement.

Billing Module

The Billing Module automates invoice generation.

Functions:

- Billing staff login
- Generate bill based on appointment
- Include doctor fee, tests, and medicines
- Select or ignore bill items
- Update bill status (Pending / Completed)
- Print bill
- Logout

The automated calculation reduces billing errors.

6.4.1 Introduction

The Hospital Management System (HMS) uses a **MySQL relational database** to store and manage all hospital-related data in a structured and secure manner.

The database is designed using Django's Object Relational Mapping (ORM) and follows normalization principles to avoid redundancy and maintain data integrity.

Each module in the system (Admin, Doctor, Patient, Laboratory, Pharmacy, Billing) is supported by corresponding database tables. The tables are interlinked using **primary keys and foreign key constraints** to maintain referential integrity.

6.4.2 List of Tables

The system consists of the following major tables:

1. Admin_details
2. Doctor_details
3. Staff_details
4. Patient_details
5. Appointment_details
6. Prescription_details
7. Medicine_details
8. Test_details
9. Test_master
10. Medicine_inventory
11. Bill

12. BillTestItem
13. BillMedicineItem
14. AppointmentOTP

Each table is described below.

6.4.3 Table Descriptions

Admin_details

Field Name	Type	Description
id	Integer (PK)	Unique admin identifier
Admin_username	Varchar	Admin login username
Admin_password	Varchar	Admin login password

This table stores administrator credentials for system access.

Doctor_details

Field Name	Type	Description
id	Integer (PK)	Unique doctor ID
Doctor_name	Varchar	Name of doctor
Doctor_username	Varchar	Login username
Doctor_password	Varchar	Login password
Doctor_phone	BigInt	Contact number
Doctor_department	Varchar	Department specialization
availability_status	Varchar	Availability status

This table maintains all registered doctor information.

Staff_details

Field Name	Type	Description
id	Integer (PK)	Unique staff ID
Staff_name	Varchar	Name of staff

Field Name	Type	Description
Staff_username	Varchar	Login username
Staff_password	Varchar	Login password
Staff_phone_no	BigInt	Contact number
Staff_user_type	Varchar	Role (Lab/Pharmacy/Billing/Receptionist)

Stores non-doctor staff details.

Patient_details

Field Name	Type	Description
id	Integer (PK)	Unique patient ID
Patient_name	Varchar	Name of patient
Patient_age	Integer	Age
Patient_gender	Varchar	Gender
Patient_phone	BigInt	Contact number
Patient_address	Varchar	Address

Stores patient personal information.

Appointment_details

Field Name	Type	Description
id	Integer (PK)	Unique appointment ID
PatientData	Foreign Key	Linked patient
DoctorData	Foreign Key	Linked doctor
Date	Date	Appointment date
Token_number	Integer	Queue number
Appointment_time	Time	Appointment time
Prescription_status	Varchar	Pending/Complete
Created_at	DateTime	Record creation time

This table connects patients and doctors.

Prescription_details

Field Name	Type	Description
id	Integer (PK)	Unique prescription ID
Appointment	Foreign Key	Related appointment
Duration	Integer	Treatment duration
Diagnosis	Varchar	Diagnosis details
Notes	Text	Additional notes
Created_at	DateTime	Creation time

Stores prescription data issued by doctors.

Medicine_details

Field Name	Type	Description
id	Integer (PK)	Unique record ID
Prescription	Foreign Key	Linked prescription
Medicine	Foreign Key	Linked inventory
Morning	Integer	Morning dosage
Afternoon	Integer	Afternoon dosage
Night	Integer	Night dosage
Instructions	Varchar	Usage instructions

Stores medicine instructions per prescription.

Test_details

Field Name	Type	Description
id	Integer (PK)	Unique test ID
Appointment	Foreign Key	Related appointment
Test_name	Varchar	Test assigned
Notes	Text	Additional remarks
Test_status	Varchar	Assigned/Completed
Report_image	ImageField	Uploaded report

Field Name	Type	Description
Created_at	DateTime	Timestamp

Manages laboratory test assignments and reports.

Test_master

Field Name	Type	Description
id	Integer (PK)	Unique test ID
Test_name	Varchar	Name of test
Price	Decimal	Test cost
Created_at	DateTime	Timestamp

Master table for laboratory test catalog.

Medicine_inventory

Field Name	Type	Description
id	Integer (PK)	Unique medicine ID
Medicine_name	Varchar	Medicine name
Medicine_price	Float	Price per unit
Stock	Integer	Stock status

Maintains pharmacy inventory.

Bill

Field Name	Type	Description
id	Integer (PK)	Unique bill ID
appointment	OneToOne FK	Linked appointment
doctor_fee	Decimal	Consultation fee
total_amount	Decimal	Total payable amount
status	Varchar	Pending/Completed
created_at	DateTime	Timestamp

Stores billing summary.

BillTestItem

Field Name	Type	Description
id	Integer (PK)	Unique record identifier
bill	Foreign Key	Linked bill record
test	Foreign Key	Linked laboratory test from Test_master
test_name_snapshot	Varchar	Test name stored at billing time
test_price_snapshot	Decimal	Test price stored at billing time

This table stores individual laboratory test charges associated with a bill. The test name and price are stored as snapshot values to preserve the billing record even if the test catalog changes later.

BillMedicineItem

Field Name	Type	Description
id	Integer (PK)	Unique record identifier
bill	Foreign Key	Linked bill
medicine	Foreign Key	Linked medicine from inventory
medicine_name_snapshot	Varchar	Medicine name stored at billing time
price_per_piece_snapshot	Decimal	Price per medicine unit
duration_days	Integer	Number of days medicine should be taken
times_per_day	Integer	Number of doses per day
quantity	Integer	Total quantity dispensed
total_price	Decimal	Total medicine cost

This table records medicine charges and prescription quantities included in a bill. Snapshot values are used to maintain billing accuracy even if medicine prices change later.

AppointmentOTP

Field Name	Type	Description
id	Integer (PK)	Unique OTP record ID
phone	BigInt	Patient phone number
otp	Varchar	Generated OTP code

Field Name	Type	Description
created_at	DateTime	OTP generation timestamp
is_used	Boolean	Indicates whether OTP has been used

This table stores OTP verification data used for secure appointment access. The OTP remains valid for 5 minutes and becomes invalid once used.

6.4.4 Relationship Between Tables

The database follows relational integrity:

- One Patient → Many Appointments
- One Doctor → Many Appointments
- One Appointment → One Bill
- One Appointment → One Prescription
- One Prescription → Many Medicines
- One Appointment → Many Tests
- One Bill → Many Test Items
- One Bill → Many Medicine Items

These relationships ensure data consistency and avoid redundancy.

6.4.5 Advantages of Database Design

- Structured relational schema
- Reduced redundancy
- Secure data storage
- Referential integrity maintained
- Scalable architecture
- Efficient querying and reporting

6.5 DATA FLOW DIAGRAMS (DFD)

A **Data Flow Diagram (DFD)** represents the flow of data through a system. It shows:

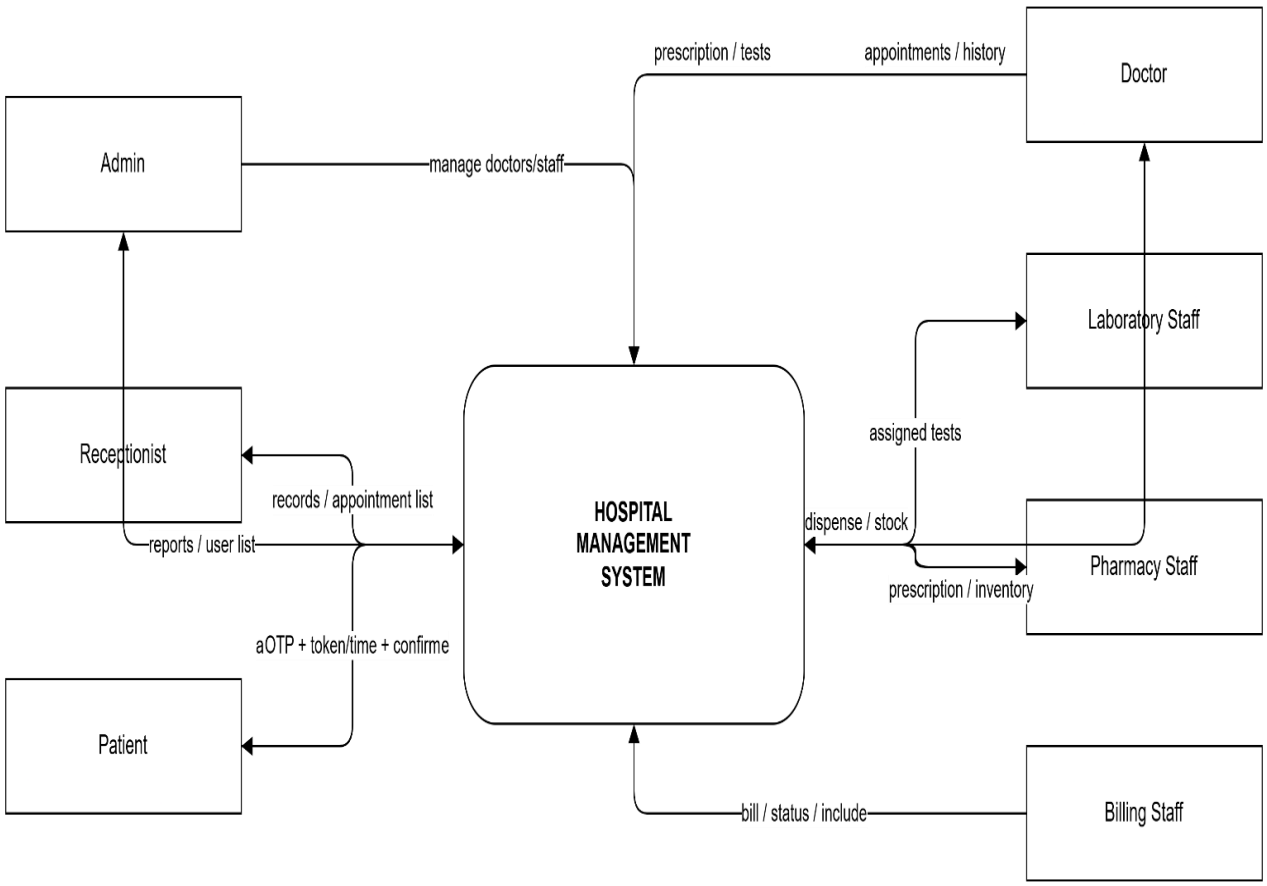
- Inputs entering the system
- Processes transforming the data
- Outputs leaving the system
- Storage of data in databases

6.5.1 DFD Symbols / Legend

- **External Entity**: User/Actor interacting with the system
- **Process**: Operation performed by the system
- **Data Store**: Database tables/storage
- **Data Flow**: Movement of data between entities/processes/stores

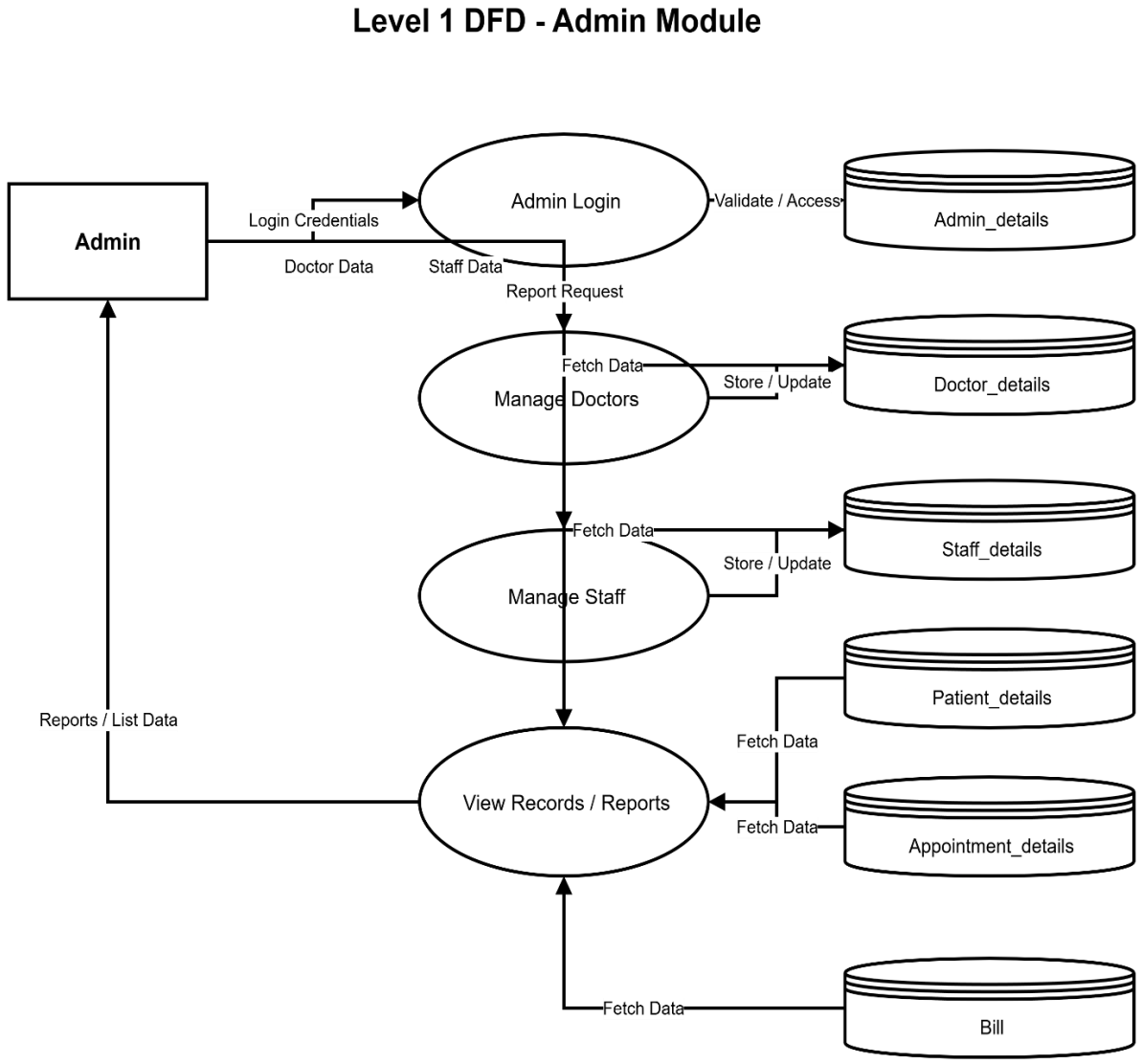
6.5.2 LEVEL 0 DFD

LEVEL 0 DFD

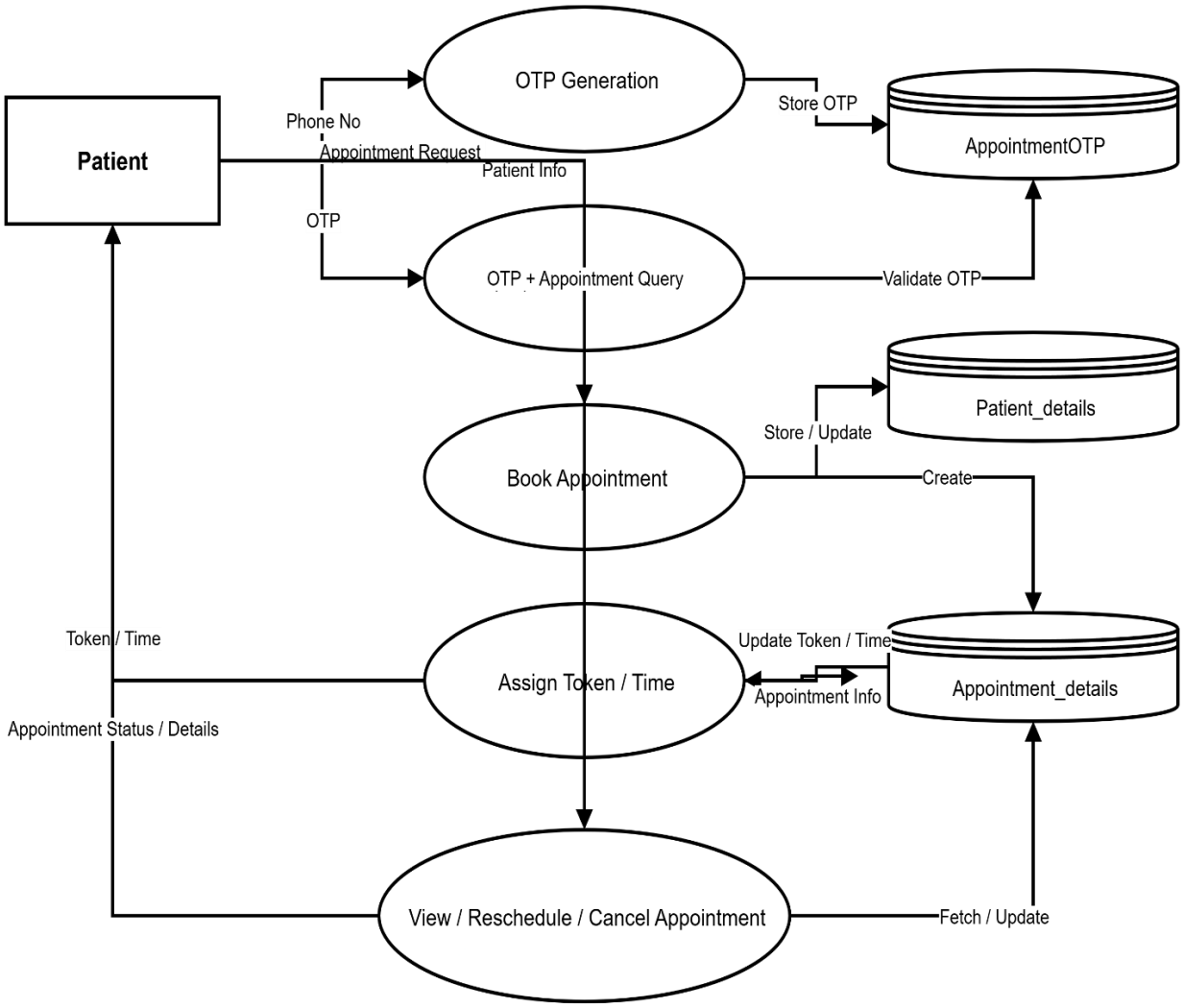


6.5.3 LEVEL 1 DFD (Module-wise)

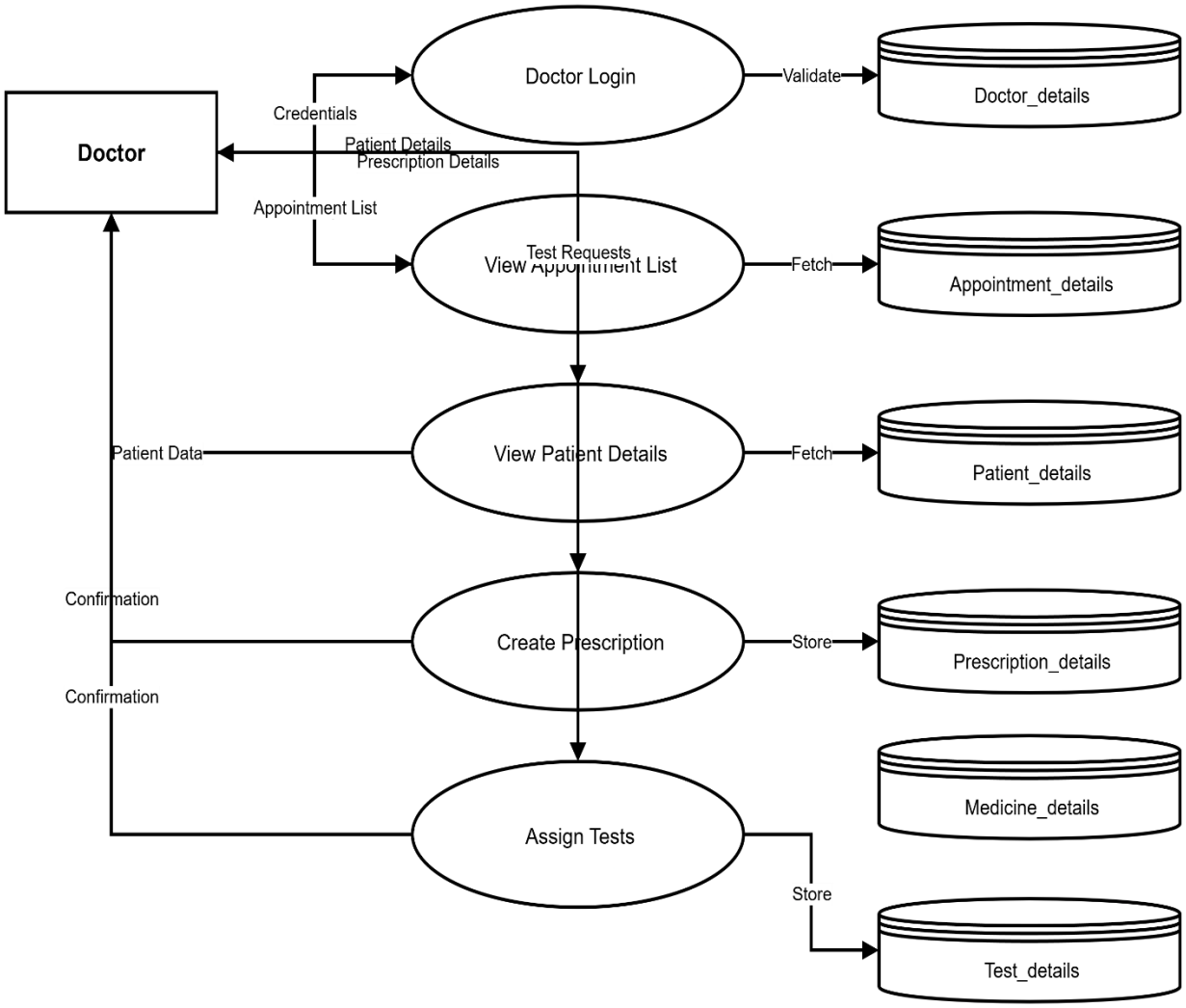
Below are Level 1 DFDs for the major modules.



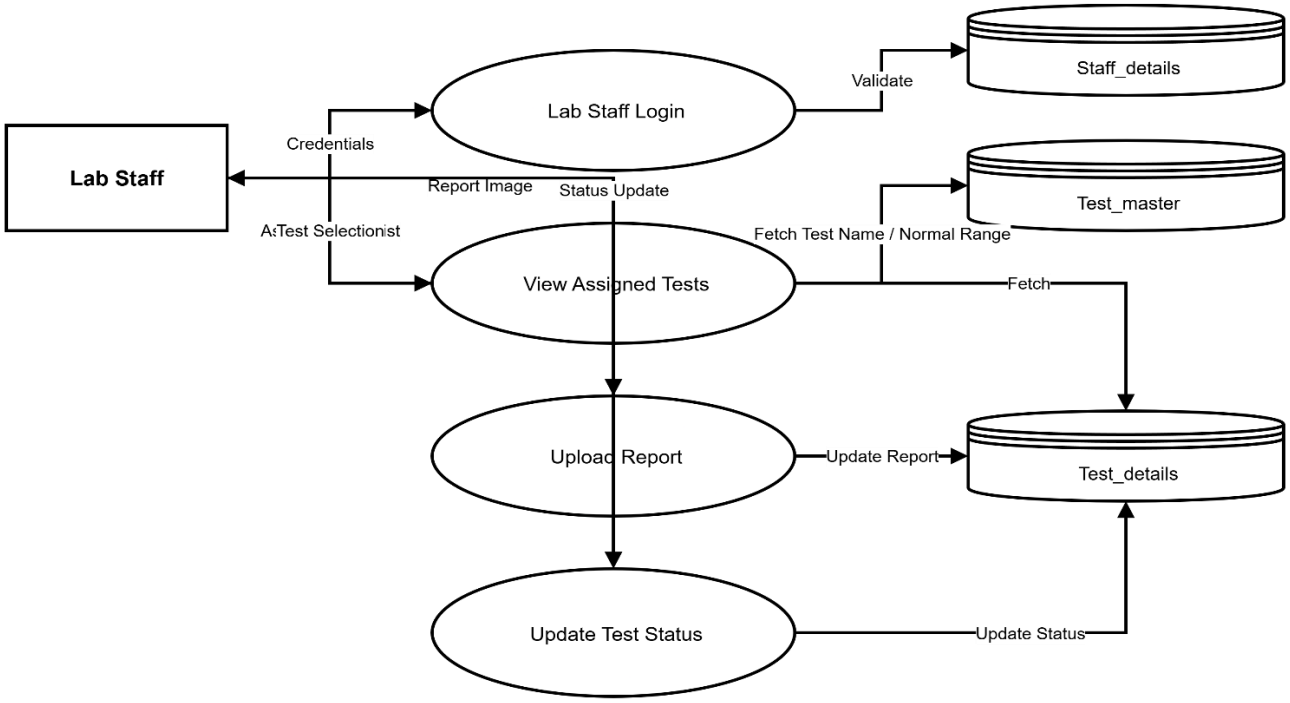
Level 1 DFD - Patient Module



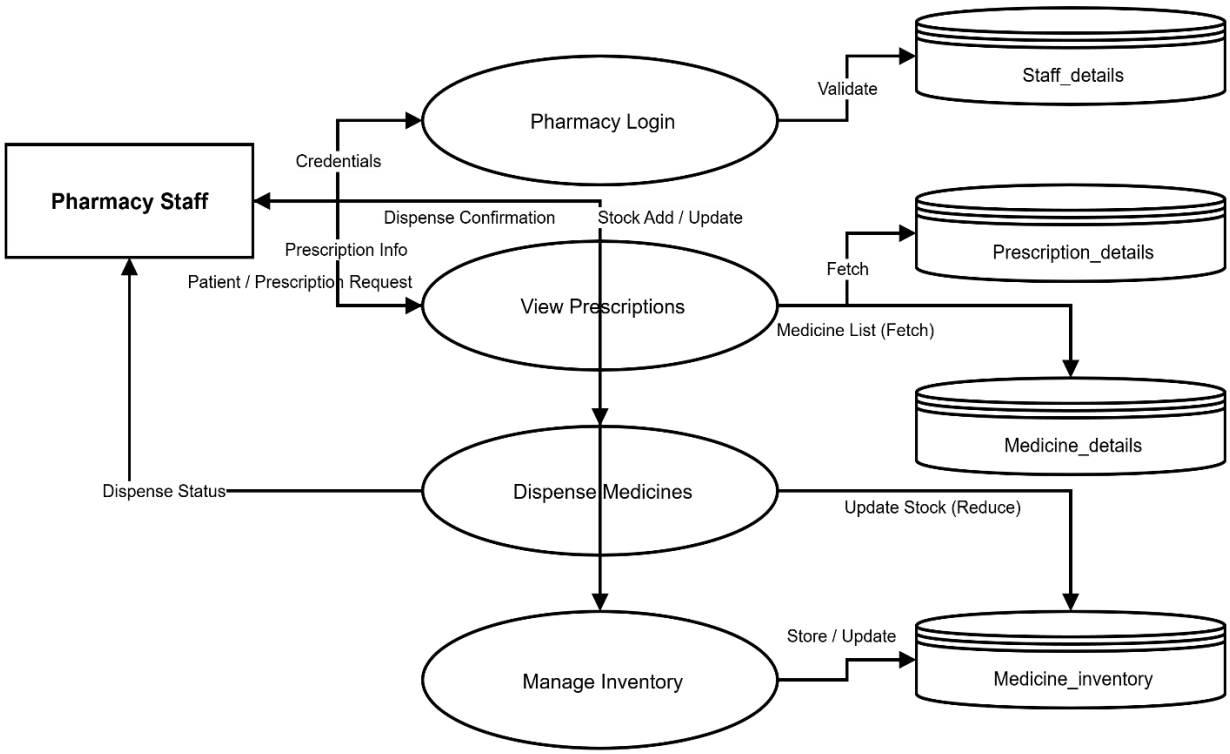
Level 1 DFD - Doctor Module



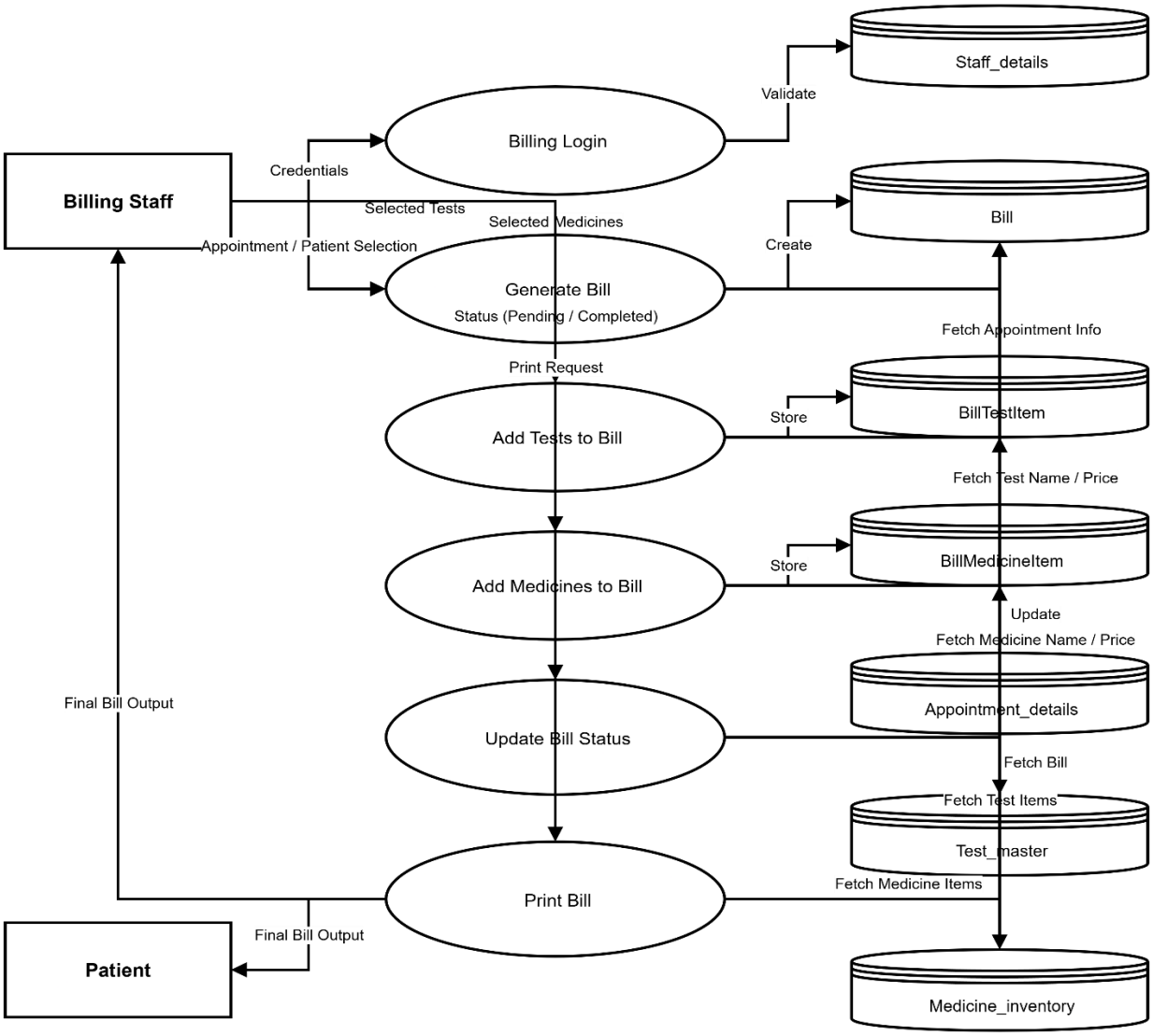
Level 1 DFD - Laboratory Module



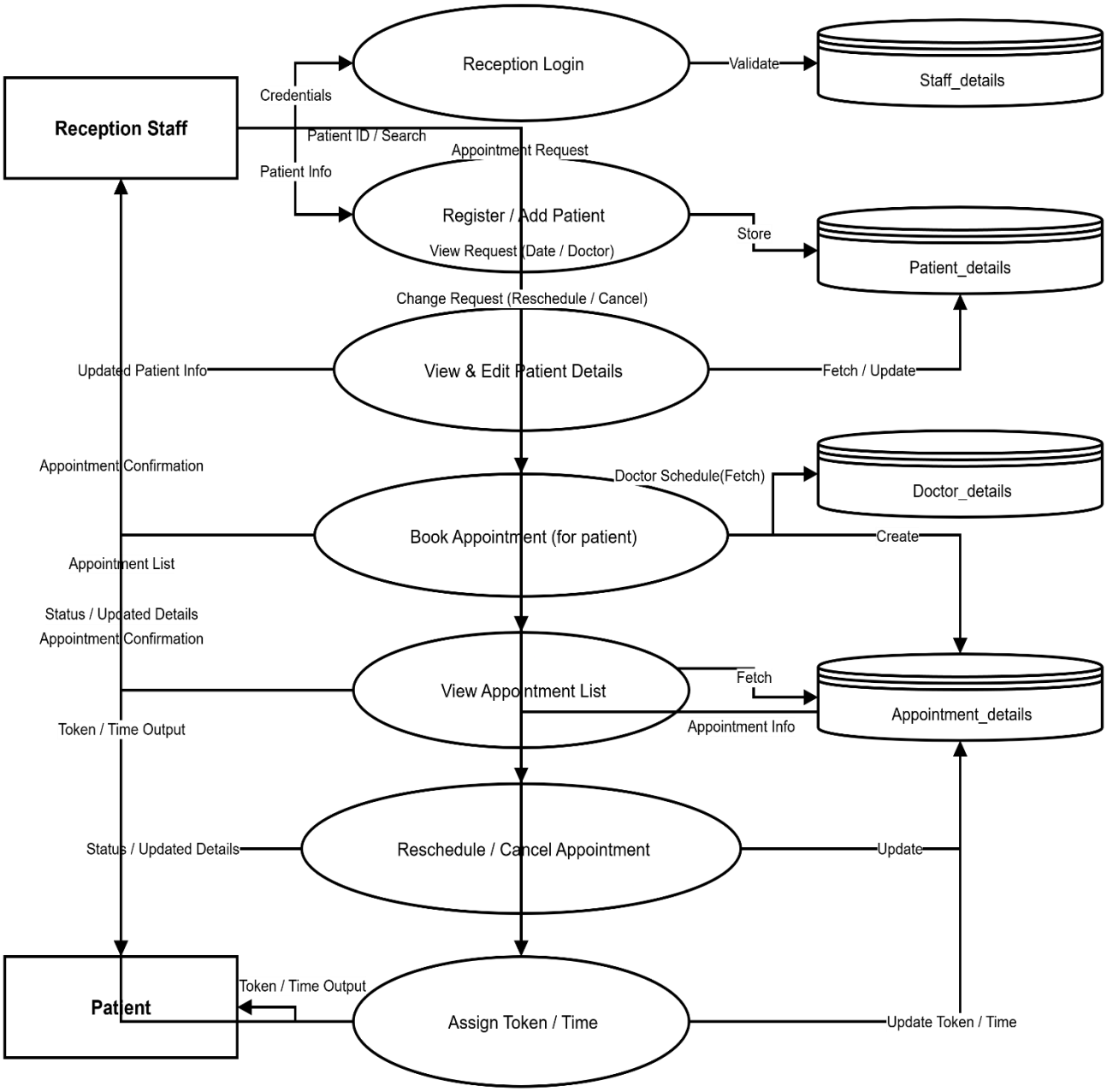
Level 1 DFD - Pharmacy Module



Level 1 DFD - Billing Module



Level 1 DFD - Reception Module



7. SYSTEM TESTING

7.1 INTRODUCTION

System testing is an important phase in software development where the complete application is tested to verify that it functions correctly according to the specified requirements. The objective of system testing is to detect errors, validate system performance, and ensure that all modules of the application work together as expected.

In the Hospital Management System (HMS), system testing was performed after the development of all modules including Admin, Patient & Appointment, Doctor, Laboratory, Pharmacy, and Billing modules. Each module was tested individually and then integrated to verify that the entire system operates smoothly.

The testing process ensures that the system is reliable, efficient, and capable of handling real hospital operations without errors.

7.2 OBJECTIVES OF SYSTEM TESTING

The main objectives of system testing in the Hospital Management System are:

• To verify that all modules work correctly according to system requirements
• To ensure that data is stored and retrieved accurately from the database
• To check proper interaction between different modules
• To identify and fix system errors before deployment
• To validate the overall performance and reliability of the system
• To ensure secure access through role-based authentication

Through systematic testing, the application becomes stable and ready for real-world usage.

7.3 TYPES OF TESTING PERFORMED

Different types of testing were performed to ensure the reliability and correctness of the system.

7.3.1 Unit Testing

Unit testing involves testing individual components or functions of the system independently. Each function and module was tested separately to verify that it performs the intended operation.

Examples of unit testing in this project include:

• Testing patient registration form validation
• Testing appointment booking functionality
• Testing prescription creation by doctors
• Testing laboratory report upload feature
• Testing billing calculation logic

This helped identify errors in individual components before integrating them with other modules.

7.3.2 Integration Testing

Integration testing verifies the interaction between different modules of the system. Since the Hospital Management System consists of multiple interconnected modules, it is important to ensure that data flows correctly between them.

Examples of integration testing include:

• Appointment data being accessible to the doctor module
• Prescriptions being visible to the pharmacy module
• Test assignments appearing in the laboratory module
• Test and medicine details being included in the billing module

This testing ensured proper communication between all system components.

7.3.3 System Testing

System testing evaluates the entire application as a complete system. It checks whether the system performs all functions according to the requirements specified during the analysis phase.

The following aspects were tested:

• User authentication for all roles (Admin, Doctor, Staff)
• Appointment scheduling and token assignment
• Prescription generation and test assignment
• Test report upload and status update
• Medicine dispensing and stock management
• Bill generation and payment status update

All modules were verified to ensure that the system functions correctly in a real hospital workflow.

7.3.4 User Interface Testing

User interface testing ensures that the system interface is user-friendly, clear, and easy to navigate.

The following aspects were tested:

• Form layouts and input fields
• Navigation between pages
• Button functionality (View, Edit, Print, etc.)
• Dashboard display for different roles

This testing ensured that hospital staff can easily interact with the system without confusion.

7.3.5 Database Testing

Database testing verifies that all system data is correctly stored, retrieved, and updated in the MySQL database.

The following checks were performed:

• Patient information stored correctly after registration
• Appointment details saved with correct token and time
• Prescription data linked with the correct appointment
• Test reports stored and retrieved successfully
• Billing records correctly calculated and stored

This testing ensures data integrity and consistency.

7.4 TEST CASES

The following table shows sample test cases performed during system testing.

Test Case ID	Module	Test Description	Expected Result	Status
TC01	Login	Admin login with valid credentials	Admin dashboard displayed	Pass
TC02	Patient Registration	Register new patient with valid details	Patient record stored successfully	Pass
TC03	Appointment Booking	Book appointment for patient	Token number and time assigned	Pass
TC04	Prescription Creation	Doctor adds prescription	Prescription stored successfully	Pass
TC05	Test Assignment	Doctor assigns laboratory test	Test appears in lab module	Pass
TC06	Report Upload	Lab staff uploads report	Test status updated to completed	Pass
TC07	Pharmacy	Dispense medicines from prescription	Inventory updated	Pass
TC08	Billing	Generate bill for appointment	Total bill calculated correctly	Pass

7.5 TESTING RESULT

All modules of the Hospital Management System were tested successfully. The testing process confirmed that the system performs all intended operations without major errors.

The system correctly handles patient registration, appointment scheduling, prescription generation, laboratory report management, pharmacy inventory handling, and billing operations. Data is accurately stored and retrieved from the MySQL database, and role-based access control ensures secure system usage.

Based on the testing results, the system is considered reliable, efficient, and suitable for practical hospital management operations.

8.SYSTYM IMPLEMENTATION

8.1 INTRODUCTION

System Implementation is the phase where the designed system is converted into a fully functional application. In this stage, the theoretical design of the Hospital Management System was transformed into a working web-based application using Django and MySQL.

The implementation involved:

- Setting up the development environment
- Configuring the database
- Creating separate modules (Django apps)
- Designing templates
- Connecting backend logic with frontend
- Testing functionality on a local server

8.2 DEVELOPMENT ENVIRONMENT SETUP

The system was developed using the following setup:

- Python 3.14.2 installed on Windows
- Django 4.2.11 framework
- MySQL database configured through XAMPP
- Local development server using Django (python manage.py runserver)

The MySQL database was created using XAMPP's MySQL service, and database credentials were configured in Django's settings.py file.

8.3 DATABASE CONFIGURATION

The database was configured using Django's ORM and MySQL backend.

Steps followed:

1. MySQL server started via XAMPP.
2. Database created manually in phpMyAdmin.
3. Database connection configured in settings.py.
4. Models defined in each Django app.
5. Database tables generated using:
    - python manage.py makemigrations
    - python manage.py migrate

Django automatically created the corresponding tables in MySQL based on the defined models.

8.4 APPLICATION STRUCTURE

The system was developed using a modular architecture. Separate Django applications were created for each major module to ensure better organization and scalability.

The project consists of the following Django apps:

- Admin Module App
- Doctor Module App
- Patient / Reception Module App
- Laboratory Module App
- Pharmacy Module App
- Billing Module App

Each app contains:

- models.py → Database schema
- views.py → Business logic
- urls.py → Routing
- templates/ → HTML pages
- forms.py (where required)

This modular structure improves maintainability and makes the system scalable for future expansion.

8.5 AUTHENTICATION IMPLEMENTATION

Instead of using Django's built-in authentication system, custom login tables were created for:

- Admin
- Doctor
- Staff (Lab, Pharmacy, Billing, Reception)

Each role has its own login validation logic. Based on user credentials, access is restricted to respective dashboards and functionalities.

This approach allowed customized role-based control tailored to hospital operations.

8.6 FRONTEND IMPLEMENTATION

The frontend was developed using:

- HTML for structure
- CSS for styling
- JavaScript for interactivity

Separate dashboard templates were created for each module. Data from the backend is dynamically rendered using Django template syntax.

Forms are used for:

- Patient registration
- Appointment booking
- Prescription creation
- Test assignment
- Billing generation
- Inventory updates

JavaScript is used for:

- Form validation
- UI interaction
- Button actions (e.g., print bill, view details)

8.7 SYSTEM EXECUTION

The system is executed locally using:

python manage.py runserver

The development server runs on:

http://127.0.0.1:8000/

Users access the system via a web browser.

Since the system is designed for local deployment, it does not require internet connectivity. However, it can be extended to a live server in the future.

8.8 IMPLEMENTATION METHODS

The system follows a direct implementation approach, where the old manual system is replaced by the computerized system after testing.

Key steps followed:

- Development and module testing
- Database validation
- Role-based access testing
- Final integration of all modules

User training can be provided to hospital staff to ensure smooth adoption.

9. CONCLUSION

The Hospital Management System developed in this project successfully addresses the limitations of traditional manual hospital management processes. By integrating all major hospital departments into a centralized digital platform, the system improves operational efficiency, accuracy, and coordination.

Technically, the system was implemented using **Django 4.2.11**, **Python 3.14.2**, and **MySQL**, following a modular architecture with separate applications for each hospital department. The relational database structure ensures data integrity and reduces redundancy, while the use of custom authentication tables enables role-based access control tailored to hospital workflows.

From a practical perspective, the system streamlines core hospital operations such as patient registration, appointment scheduling, digital prescription generation, laboratory test management, pharmacy inventory handling, and automated billing. The OTP-based appointment verification enhances security without requiring permanent patient login accounts.

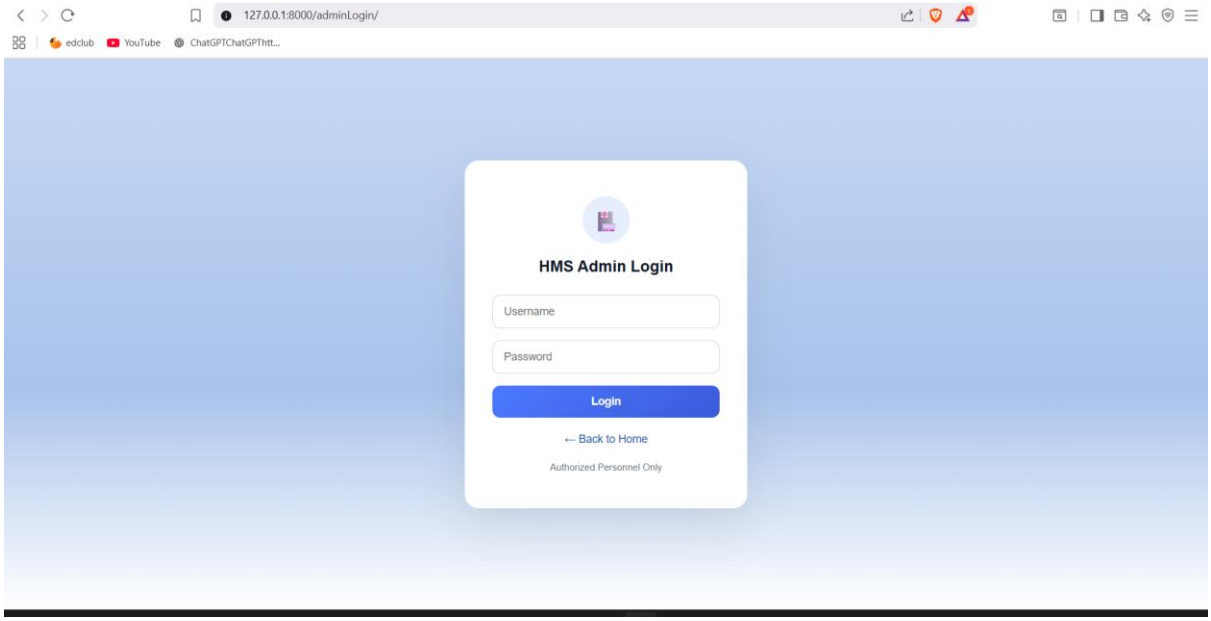
The implementation of digital prescriptions eliminates errors caused by illegible handwriting. Real-time test report uploads improve coordination between doctors and laboratory staff. Automated billing reduces calculation errors and enhances financial transparency. Pharmacy inventory management ensures better stock monitoring and prevents medicine mismanagement.

The system also significantly reduces patient waiting time and administrative workload, thereby improving overall hospital productivity and patient satisfaction.

All modules were tested and integrated successfully, and the system performs according to the defined objectives. The Hospital Management System is stable, scalable, and ready for practical deployment with minor enhancements if required.

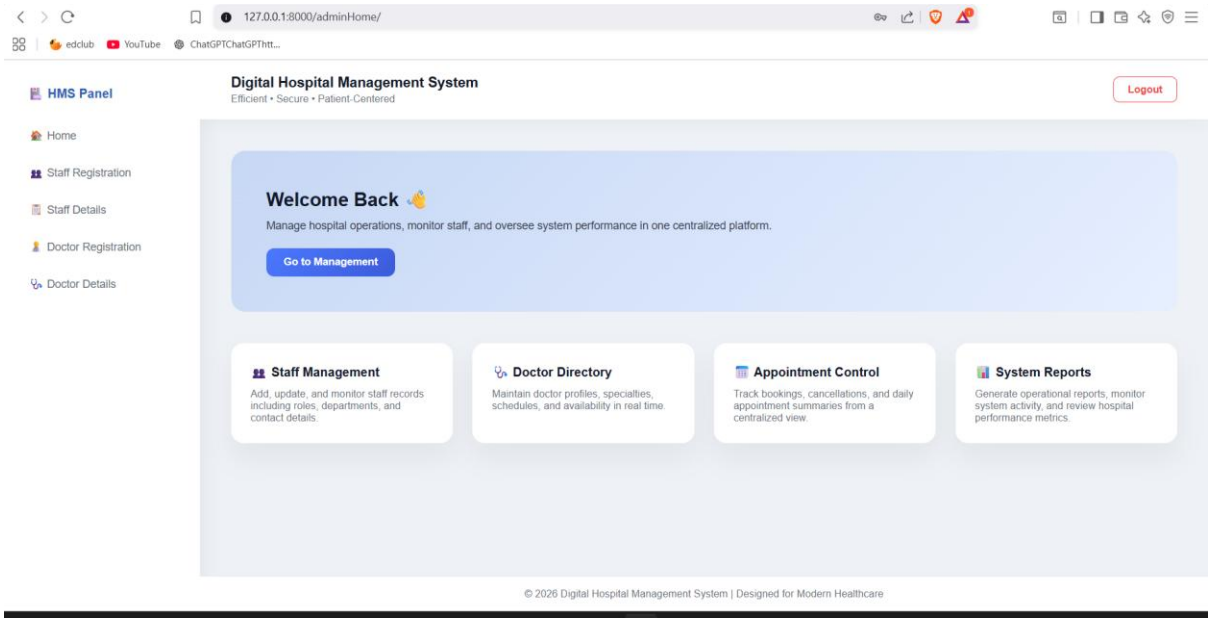
10. SYSTEM SCREENSHOTS

10.1 – Admin Login Page



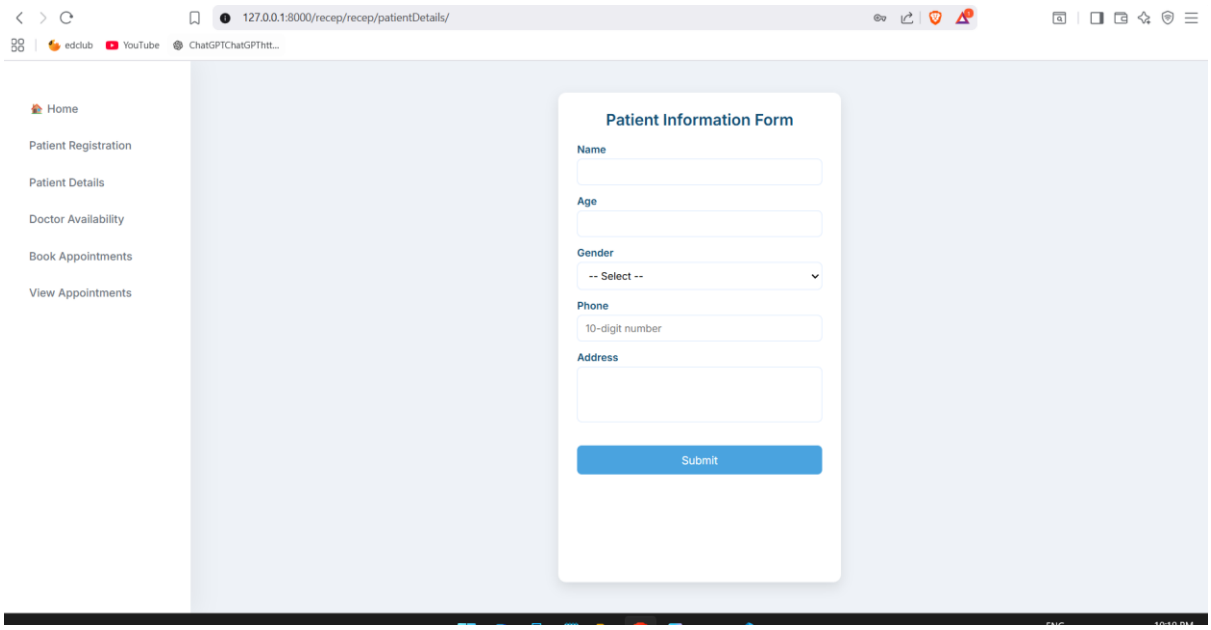
Description:
This figure shows the Admin login interface where the administrator enters valid credentials to access the system dashboard.

10.2 – Admin Dashboard



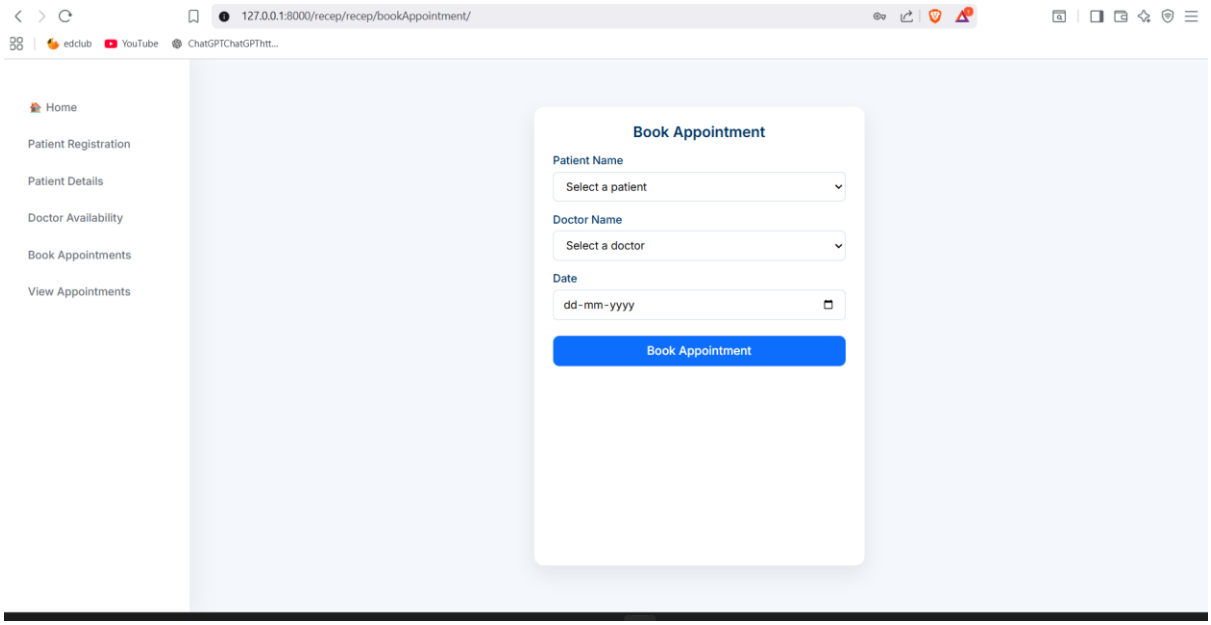
Description:
This dashboard allows the administrator to manage doctor accounts, staff accounts, and view system data.

10.3 – Patient Registration Page



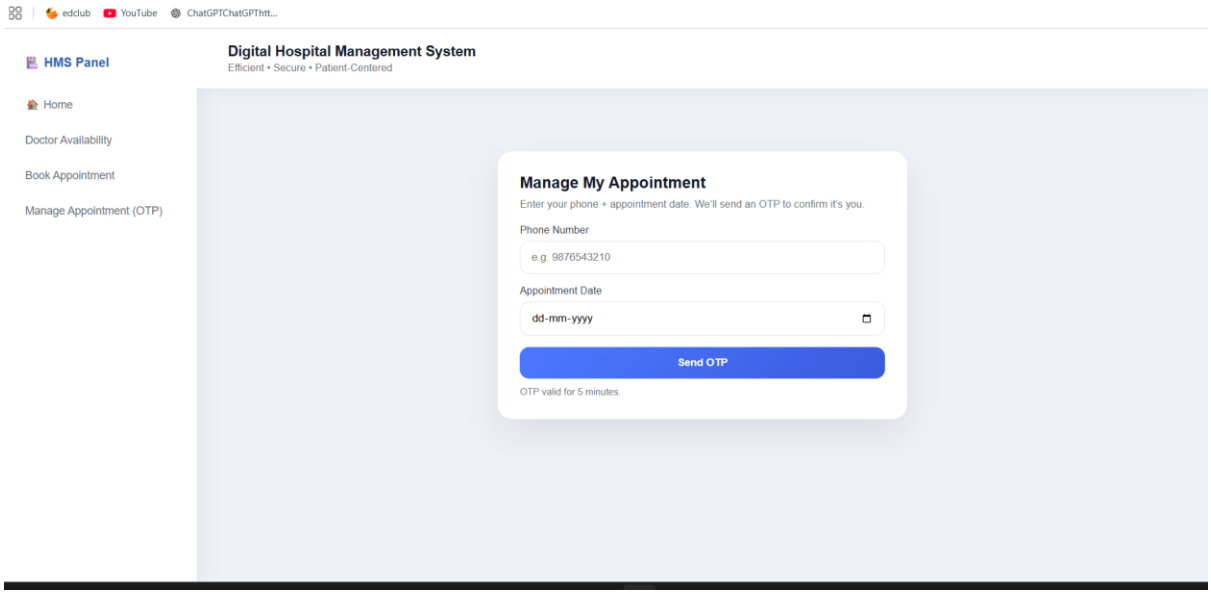
Description:
This page allows patients or reception staff to register new patients into the system by entering required details.

10.4 – Appointment Booking Page



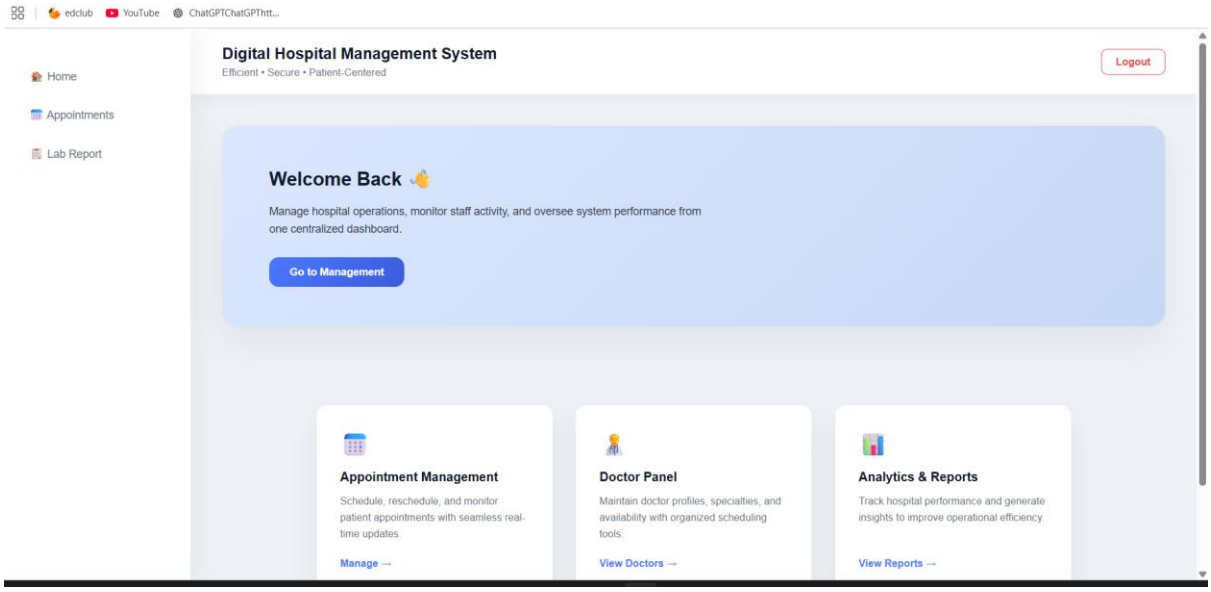
Description:
This interface allows patients to book appointments by selecting a doctor and date. Token number and appointment time are allocated.

10.5 – OTP Verification Page



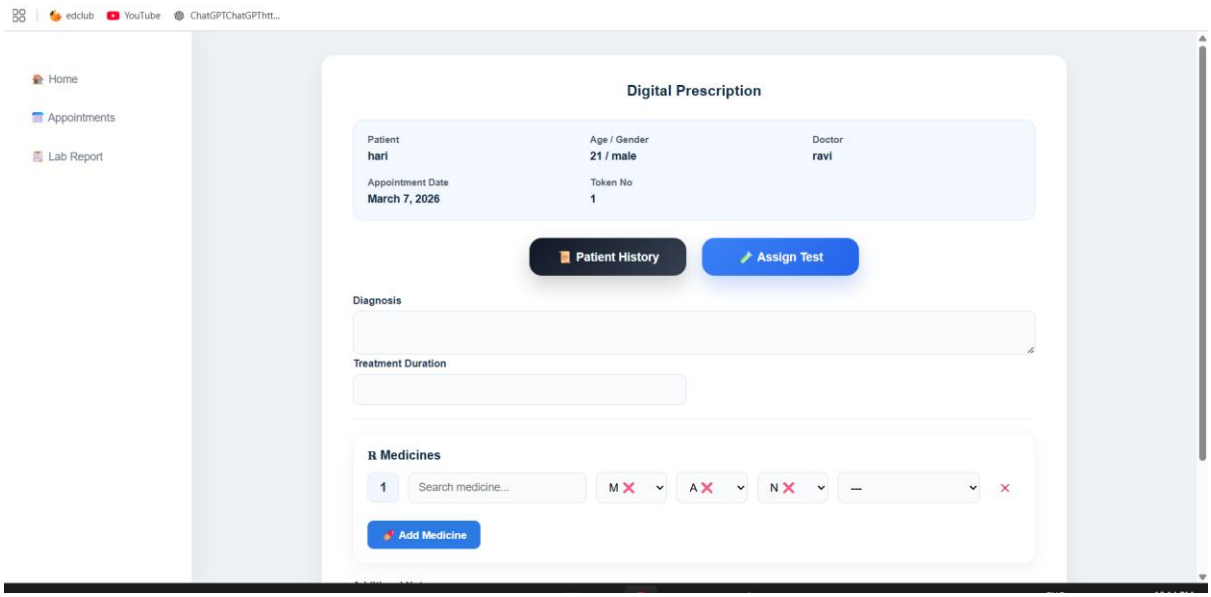
Description:
This page verifies patient identity using a one-time password (OTP), which expires after 5 minutes for security purposes.

10.6 – Doctor Dashboard



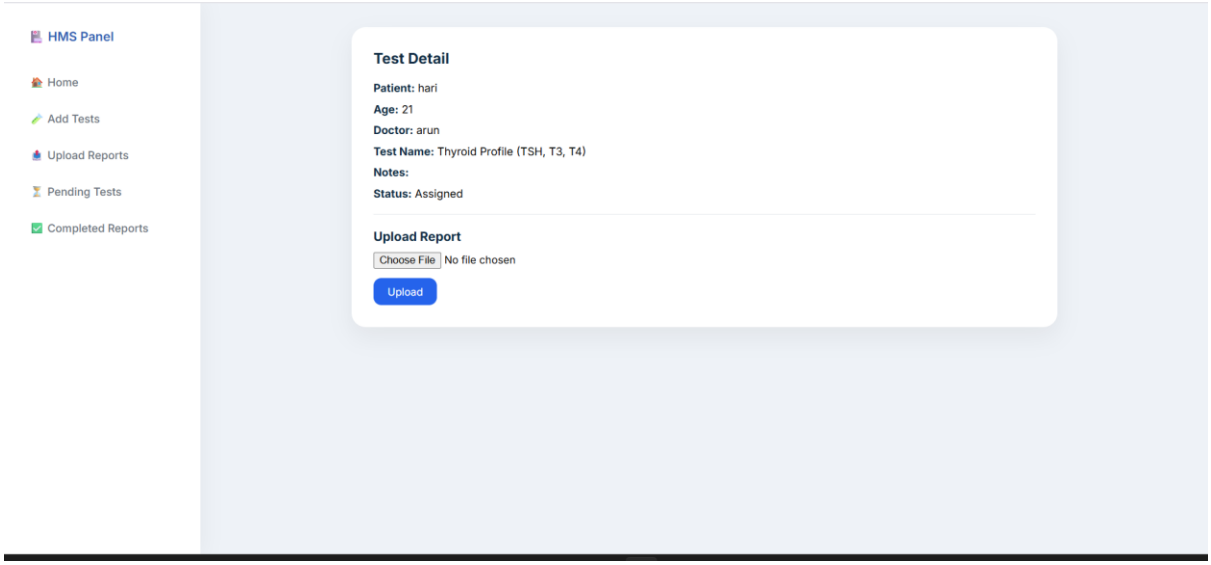
Description:
This dashboard displays scheduled appointments for the doctor and allows access to patient records.

10.7 – Prescription Creation Page



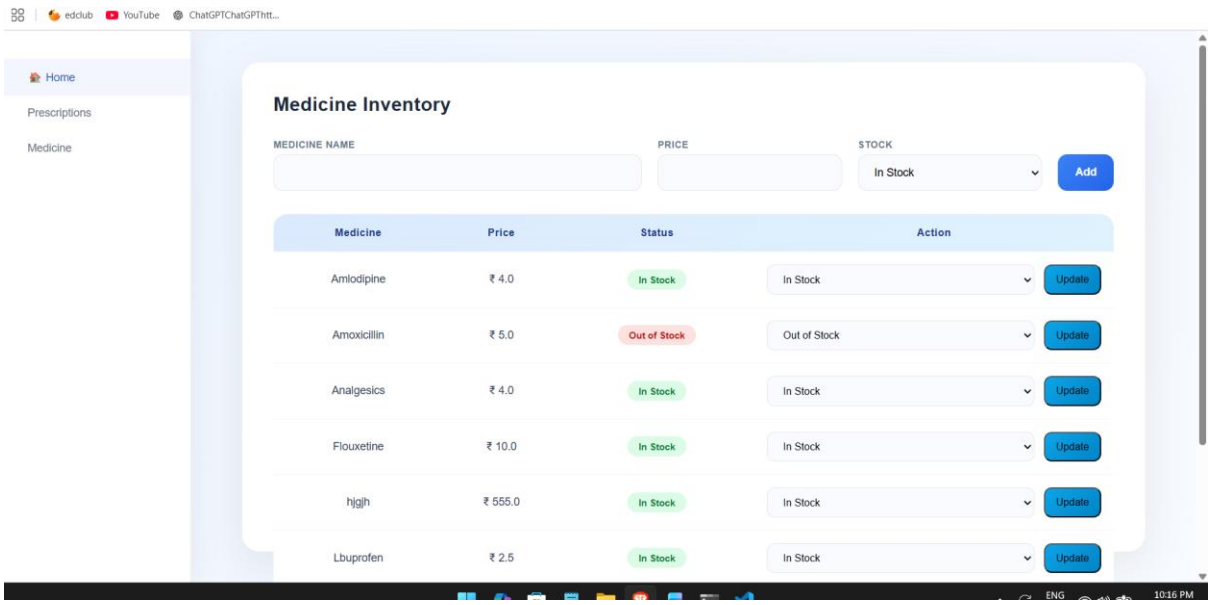
Description:
This page enables doctors to enter diagnosis details, treatment duration, medicines, and assign laboratory tests.

10.8 – Laboratory Test Management Page



Description:
Laboratory staff can view assigned tests, upload report images, and update test status.

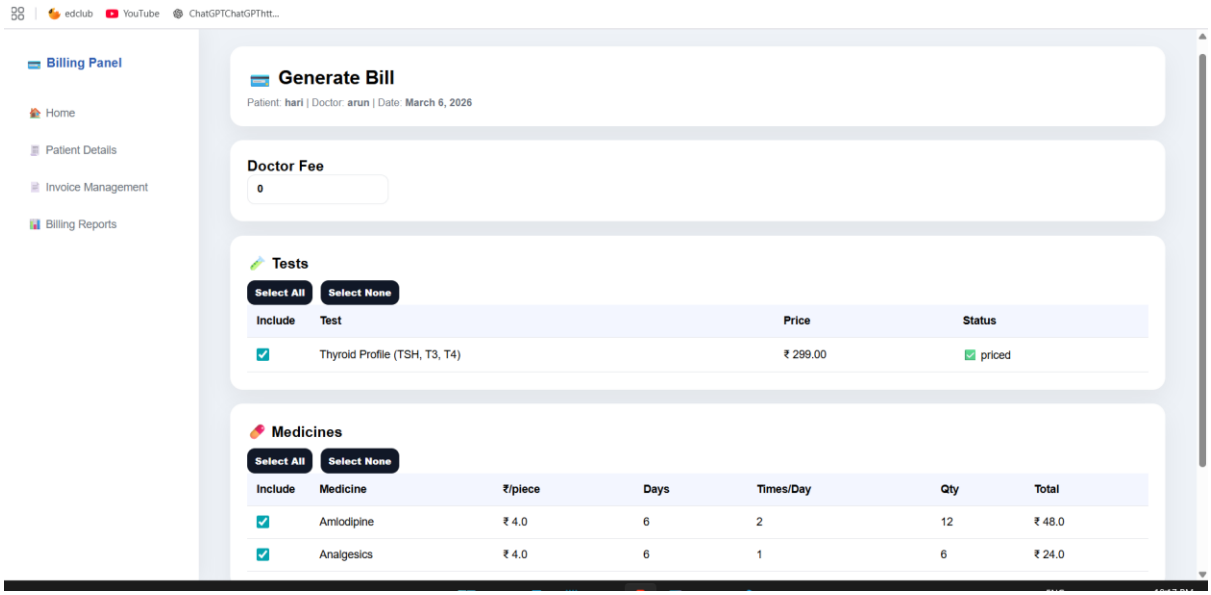
10.9 – Pharmacy Inventory Page



Description:

This page allows pharmacy staff to manage medicine inventory and update stock status.

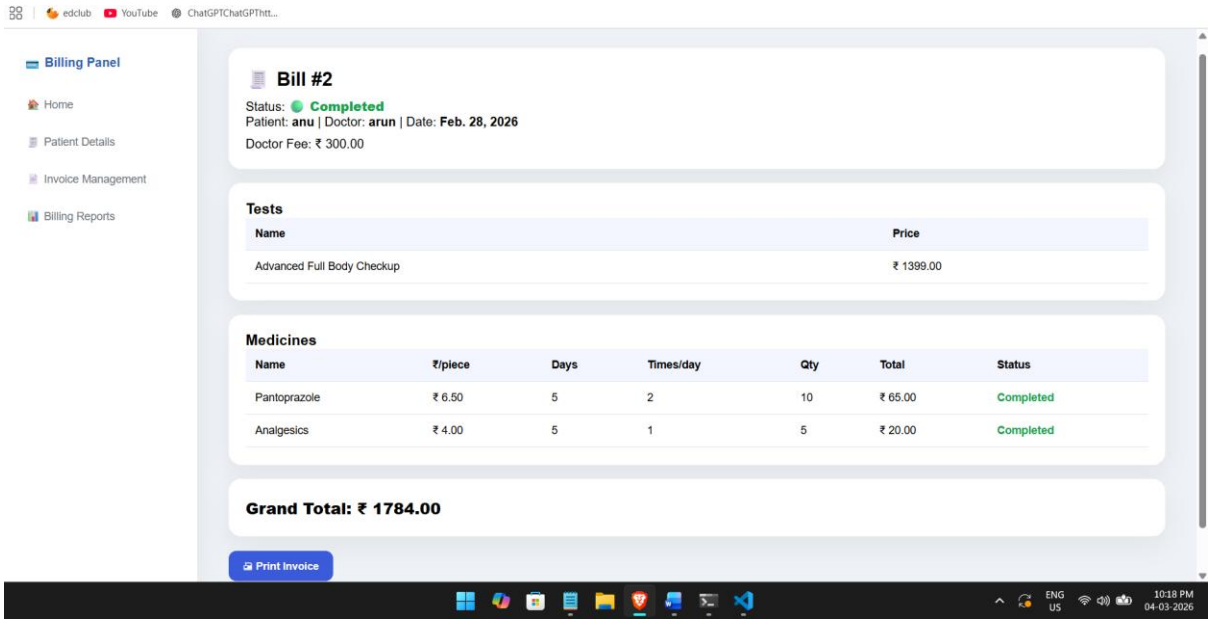
10.10 – Billing Generation Page



Description:

This interface enables billing staff to generate invoices by selecting doctor fees, test charges, and medicines.

10.11 – Generated Bill (Print View)



Description:

This figure displays the final bill format, including total calculation and payment status update option.

11. FUTURE SCOPE

Although the current system meets the core objectives of hospital management, several enhancements can be implemented in future versions to further improve functionality and scalability.

Future improvements may include:

- Deployment on a cloud server for remote access
- Integration of Django's built-in secure authentication system
- Online payment gateway integration for digital billing
- SMS or email notifications for appointment reminders
- Advanced reporting and analytics dashboard
- Role-based audit logs for enhanced security monitoring
- Integration with barcode-based pharmacy management
- Multi-branch hospital support
- Mobile application version for doctors and patients

Additionally, the system can be extended to support:

- Electronic Medical Record (EMR) standard compliance
- Real-time patient tracking
- AI-based appointment optimization
- Data encryption for enhanced security

With these enhancements, the Hospital Management System can evolve into a full-scale enterprise-level healthcare management solution.

12.REFERENCES

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2015.

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[3] Django Software Foundation, "Django Documentation," Available: https://docs.djangoproject.com/ (Accessed: Feb. 2026).

[4] Oracle Corporation, "MySQL 8.0 Reference Manual," Available: https://dev.mysql.com/doc/ (Accessed: Feb. 2026).

[5] Python Software Foundation, "Python 3 Documentation," Available: https://docs.python.org/3/ (Accessed: Feb. 2026).

[6] W3C, "HTML and CSS Standards," Available: https://www.w3.org/standards/webdesign/ (Accessed: Feb. 2026).

[7] Mozilla Developer Network (MDN), "JavaScript Guide," Available: https://developer.mozilla.org/ (Accessed: Feb. 2026).